PORTFOLIO PROJECT — ELASTIC + REDHAT OPENSHIFT AI

Building Smarter AI Apps

From Prototype to Production

A hands-on engineering lab in which I took a skeleton police incident search application — one with a working interface but no intelligence behind it — into a fully production-ready AI system. That meant loading real data, building smarter search, connecting a language model, and creating an AI assistant that analysts could actually have a conversation with.

Stack: Elasticsearch · ELSER / Sparse Embeddings · FastAPI (Python) · React · IBM Granite 3.3 LLM · Kibana Agent Builder · ES|QL · Docker

Summary: End-to-End Pipeline

Six sequential phases, each unlocking new product capabilities:

01	**Schema Design**	Designed the Elasticsearch index mapping with keyword fields for filtering, text fields for BM25 full-text search, semantic_text for automatic ELSER embedding generation, and geo_point for map visualization. Schema decisions here determined every feature available downstream.
02	**Bulk Data Ingestion**	Loaded 1,000 police incident reports via the _bulk API using a Python script with batches of 200 documents. The semantic_text field auto-triggered ELSER to generate vector embeddings at index time — no external API, no additional cost.
03	**Hybrid Search**	Built a multi-stage search pipeline in Kibana Dev Tools: BM25 keyword search, ELSER semantic search, then combined both using Reciprocal Rank Fusion (RRF). Added filter clauses and a cross-encoder reranker for production-quality relevance.
04	**App Deployment**	Deployed the full-stack application via Docker Compose. Then implemented the /search/hybrid Python endpoint, translating the Dev Tools RRF query into FastAPI code and unlocking the Hybrid button in the UI.
05	**LLM Integration**	Connected IBM Granite 3.3 8B (hosted on RedHat OpenShift AI) to Elasticsearch via the Inference API using the OpenAI-compatible vLLM endpoint. Tested completion, entity extraction, severity classification, and RAG-style prompts.
06	**RAG Pipeline**	Hand-coded the Retrieve → Augment → Generate pattern: fetch document from ES, build a structured prompt with incident context, call Granite via es_client.inference.inference(). This powered the live Generate Summary button.

07	**AI Investigation Agent**	Built a Police Investigation Assistant in Kibana Agent Builder with 5 ES|QL tools (stats, district analysis, timeline, high-value incidents, exact ID lookup) and 1 semantic Index Search tool. Tested multi-turn conversations and compared Llama-4 vs. Claude for tool-call reliability. Integrated the agent into the app via the Agent Builder API.

Starting Point — The Skeleton App

The app looked great but didn't do much. The search bar only matched exact words, the database was empty, every AI button was greyed out, and the chat widget in the corner was a placeholder that did nothing. The goal was to wire up everything behind the scenes so all of it actually worked.

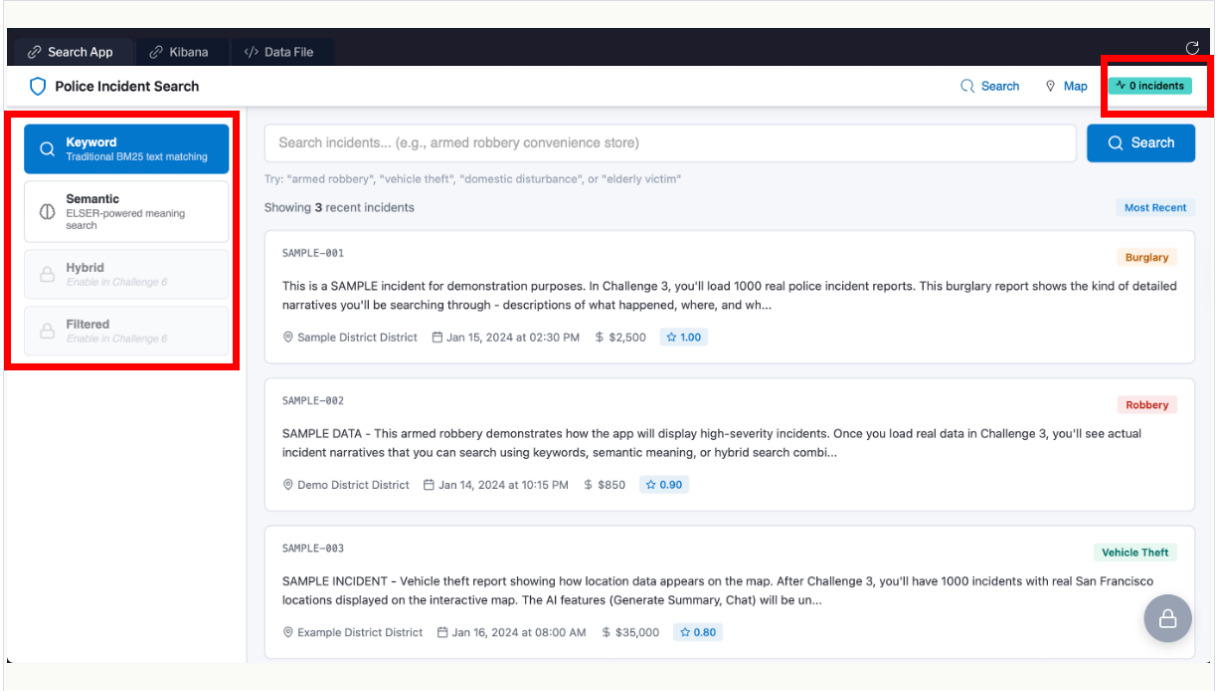


Fig. 1 — Skeleton app on day one: Hybrid and Filtered locked, 0 incidents, no LLM connected

Phase 1 — Designing the Data Structure

Before loading any data, I had to define how it would be stored — what fields exist, what types they are, and how they can be searched. This is called an index mapping, and it's one of the most important decisions in the project. Every search filter, AI feature, and agent tool we built later depended on choices made here.

Field	Type	Purpose
`narrative`	`text`	Powers regular keyword search (finds exact words)
`narrative_semantic`	`semantic_text`	Powers meaning-based search — AI embeddings generated automatically, no external service needed
`incident_type`	`keyword`	Used for filtering results and running analytics

location	geo_point	Displays incidents on the map
incident_datetime	date	Enables date range filtering and timeline views
district	keyword	Used by the AI agent to look up incidents by area
estimated_loss	float	Lets the agent find and rank high-value incidents

Phase 2 — Loading 1,000 Incidents with AI Embeddings

I loaded all 1,000 police incident reports into Elasticsearch using a Python script that sent them in batches of 200 at a time. The smart part: because the data structure included a special "semantic text" field, Elasticsearch automatically generated AI embeddings for each record as it was saved — no separate step, no third-party API, no extra cost.

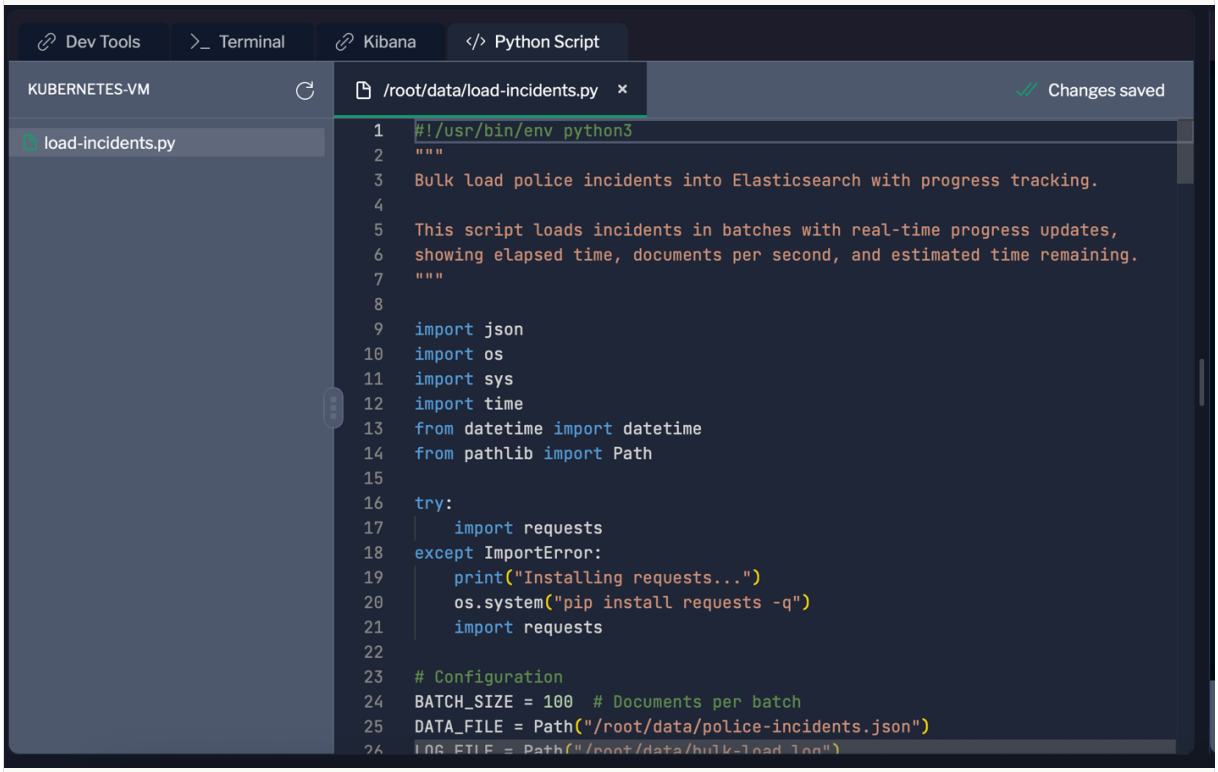


Fig. 2 — Python bulk-load script: batched ingestion with real-time progress, docs/sec, and ETA

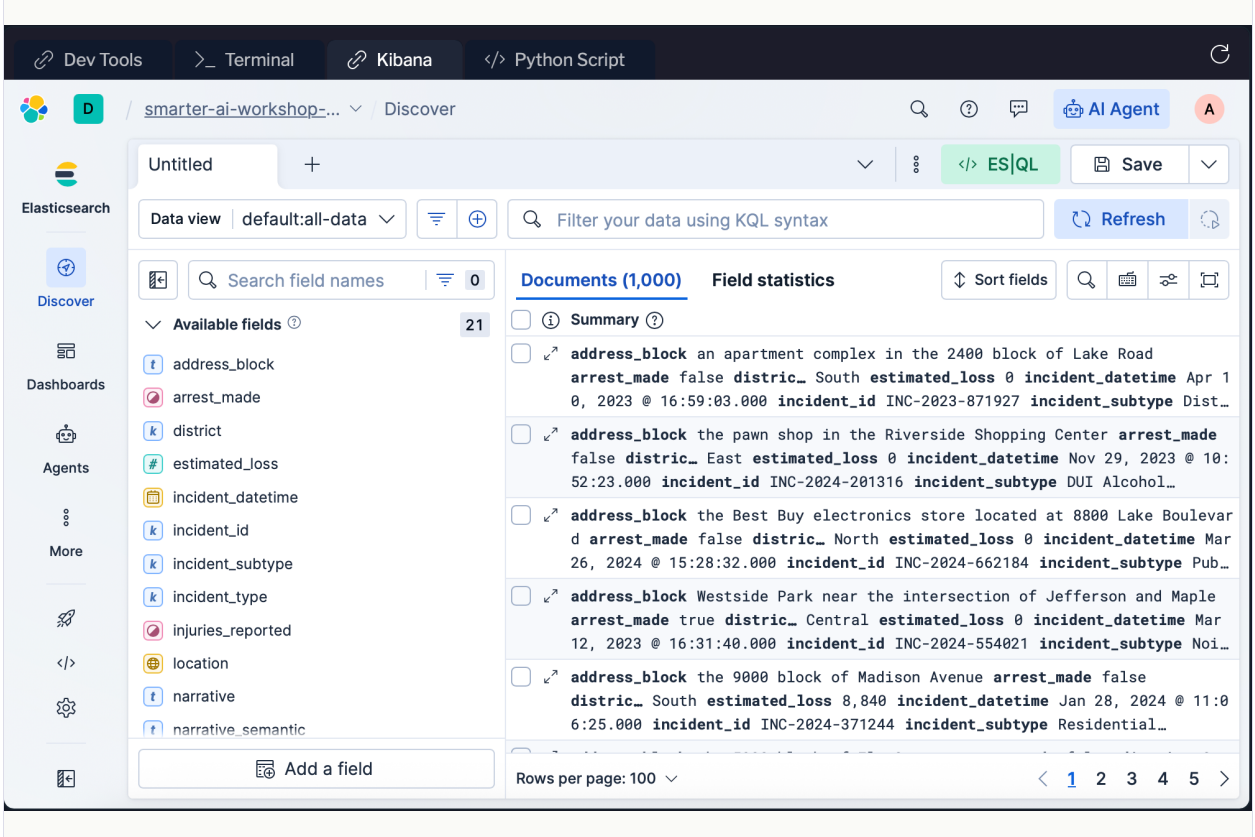


Fig. 3 — Kibana Dev Tools confirming document INC-2024-001234 indexed with 200 OK

Phase 3 — Building Smarter Search

Search was built up in layers. First, basic keyword search — finds records containing the exact words you type. Then semantic search — finds records based on meaning, even if the words don't match. Then both combined using a technique called Reciprocal Rank Fusion (RRF), which blends the two result lists in a principled way without needing to manually tune any weights.

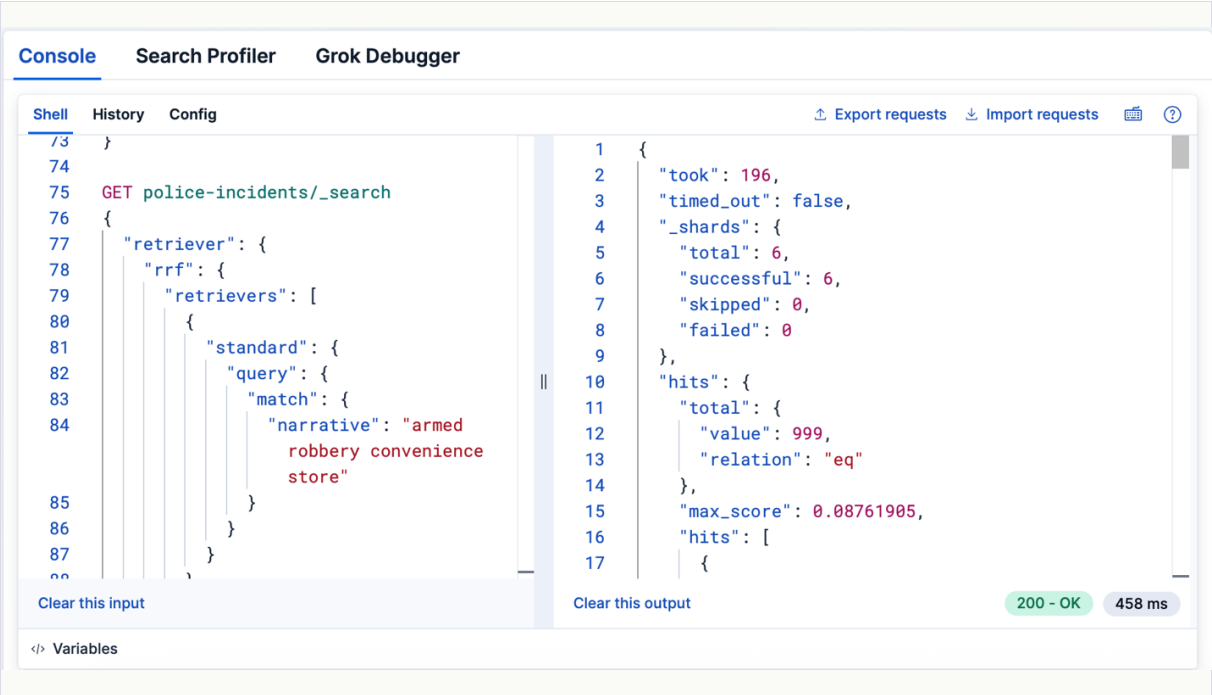


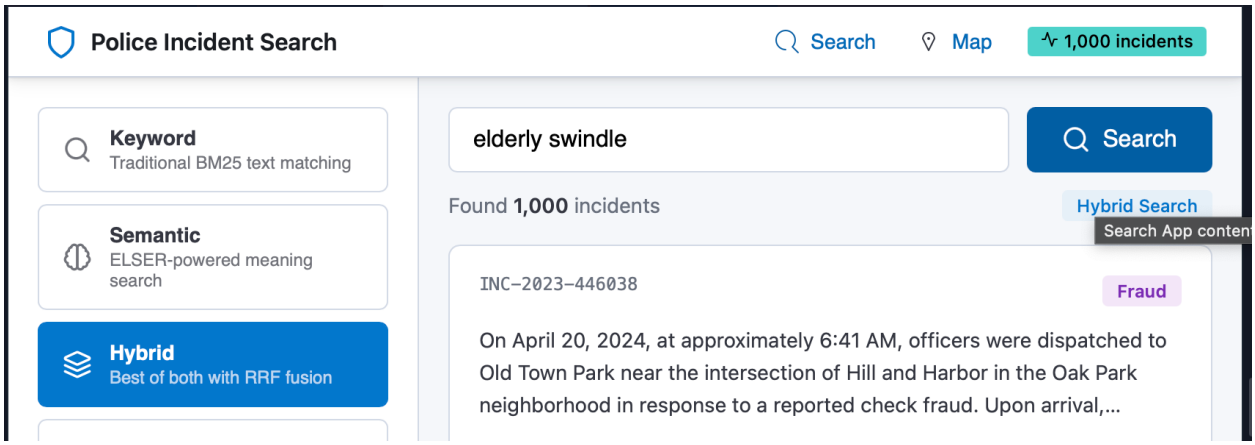
Fig. 4 — RRF retriever query in Dev Tools: rank_window_size 50, 999 hits, 200 OK in 458ms

Real-world example: Searching for “elderly swindle” with keyword search returned nothing — those exact words don’t appear in any report. With meaning-based search, it surfaced fraud cases involving elderly victims. With hybrid search, it returned the best of both (see images below).

Keyword:

Semantic:

Hybrid:



Phase 4 — Deploying the App & Unlocking Hybrid Search

The full app was launched using Docker, which packages the frontend and backend together and starts them with a single command. I then wrote the Python code for the hybrid search endpoint — translating the search logic I'd built and tested in Kibana into a real API route. This is what unlocked the "Hybrid" button in the UI.

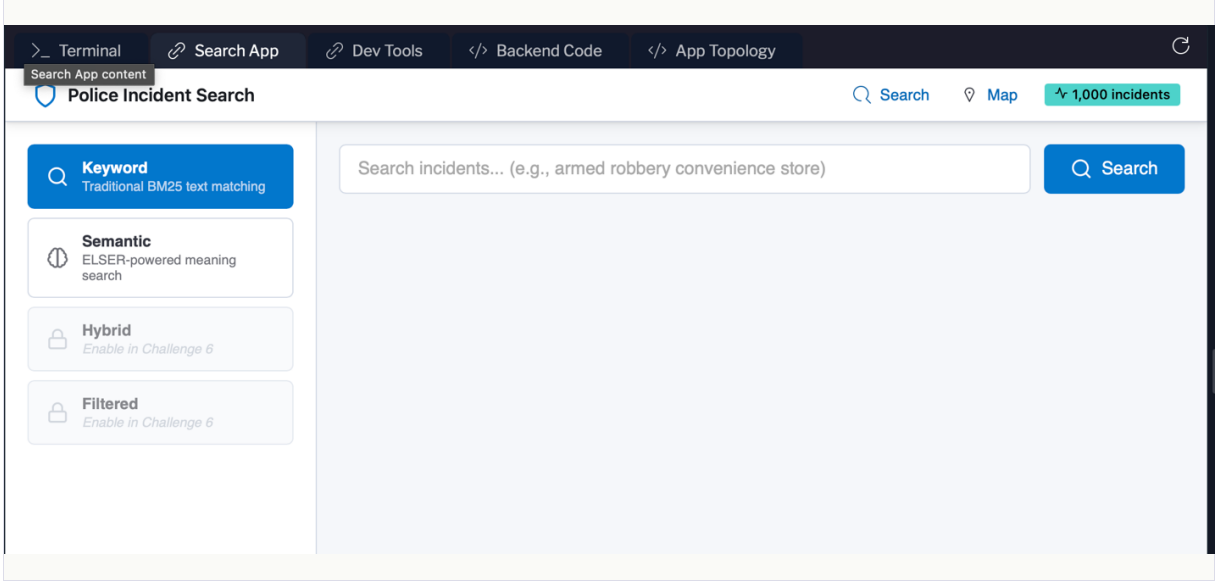


Fig. 5 — App live with 1,000 incidents loaded; Keyword and Semantic active

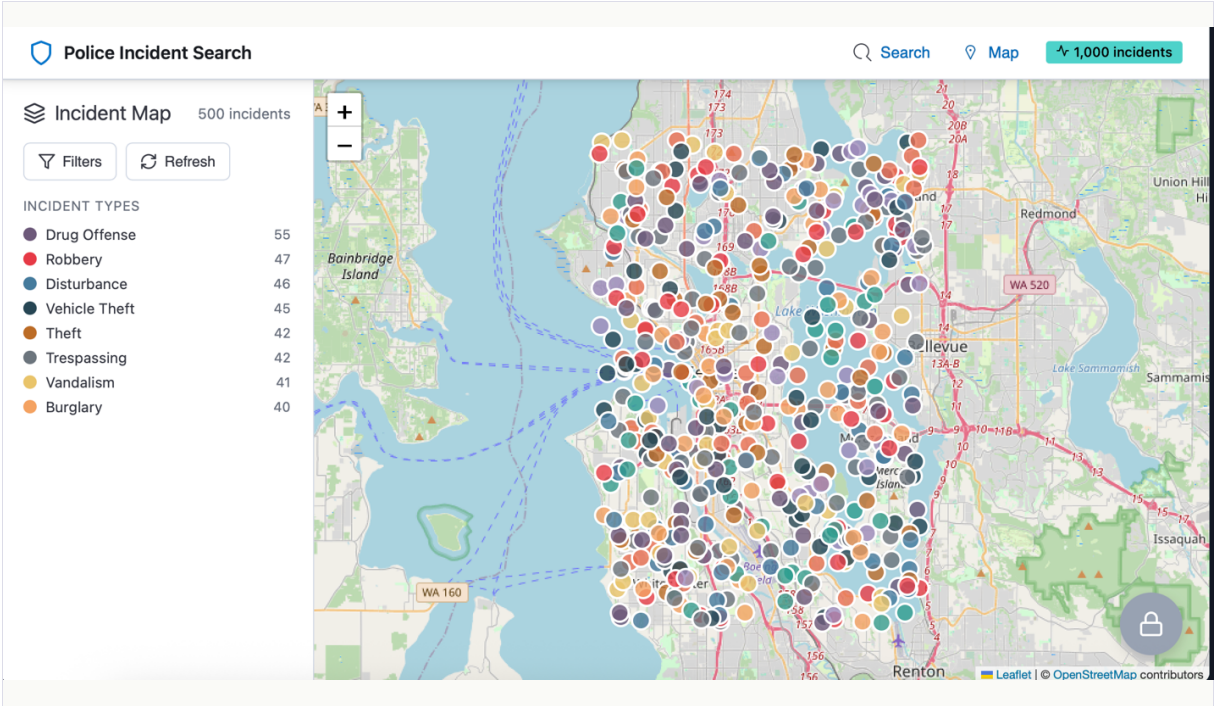


Fig. 6 — Geo_point data drives the interactive incident map, color-coded by incident type. This shows how Elasticsearch data can power different visualization types.

Phase 5 — Connecting an AI Language Model & Building RAG

I connected IBM Granite 3.3 — a large language model running on RedHat's infrastructure — to the app through Elasticsearch's built-in connector system. Then I wrote a simple three-step pipeline to make the "Generate Summary" button work: (1) fetch the incident from the database, (2) build a prompt that includes the incident details, (3) send it to the language model and return its response. This pattern is called RAG — Retrieval-Augmented Generation.

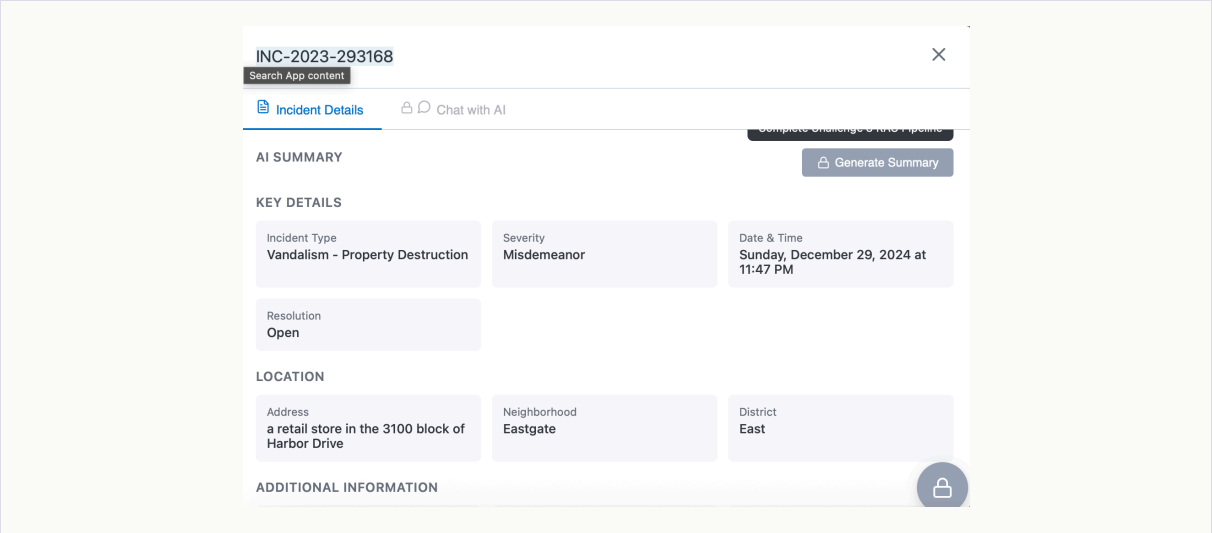


Fig. 7 — Incident detail view before RAG: Generate Summary locked

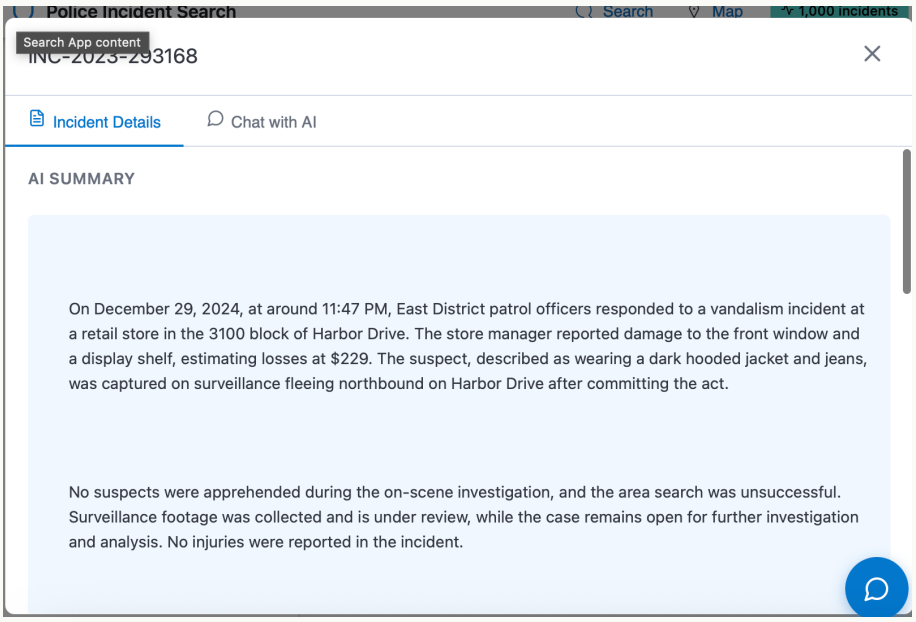


Fig. 8 — Live AI summary generated by IBM Granite via the RAG pipeline

Code Spotlight — RAG Pipeline (Challenge 8)

The RAG pattern was implemented from scratch with no abstraction libraries. Three focused async functions, each owning one responsibility:

R	**Retrieve** — es_client.get() fetches the incident document by ID, excluding the large embedding field for efficiency.
A	**Augment** — An f-string prompt assembles clear LLM instructions + key structured fields + the full narrative text.
G	**Generate** — es_client.inference.inference() calls IBM Granite through the same ES client used for search — one SDK, two capabilities.

rag.py — Retrieve → Augment → Generate

```
# RETRIEVE
async def retrieve_document(es_client, index, doc_id):
    doc = await es_client.get(
        index=index, id=doc_id,
        source_excludes=["narrative_semantic"],
    )
    return doc

# AUGMENT
def build_prompt(document):
    return f"""Summarize this incident in 2-3 bullets.
    Incident Type: {document.get('incident_type')}
    Location: {document.get('address_block')}, {document.get('district')}
District
    Narrative: {document.get('narrative')}
    Provide a brief summary:"""
```

```
# GENERATE
async def generate_summary(es_client, inference_id, prompt):
    return await es_client.inference.inference(
        model_id=inference_id,
        task_type="completion",
        input=prompt,
    )
```

Phase 6 — Building an AI Investigation Assistant

The final step was building a conversational AI agent that analysts could ask real questions — like "What are the most common crimes in the Central district?" or "Find incidents with losses over $10,000." I built it in Kibana's Agent Builder tool and gave it 6 capabilities: five structured query tools for things like counting incidents by area, viewing timelines, and looking up specific case numbers — plus one meaning-based search tool for open-ended narrative questions. The agent decides which tool to use based on what you ask.

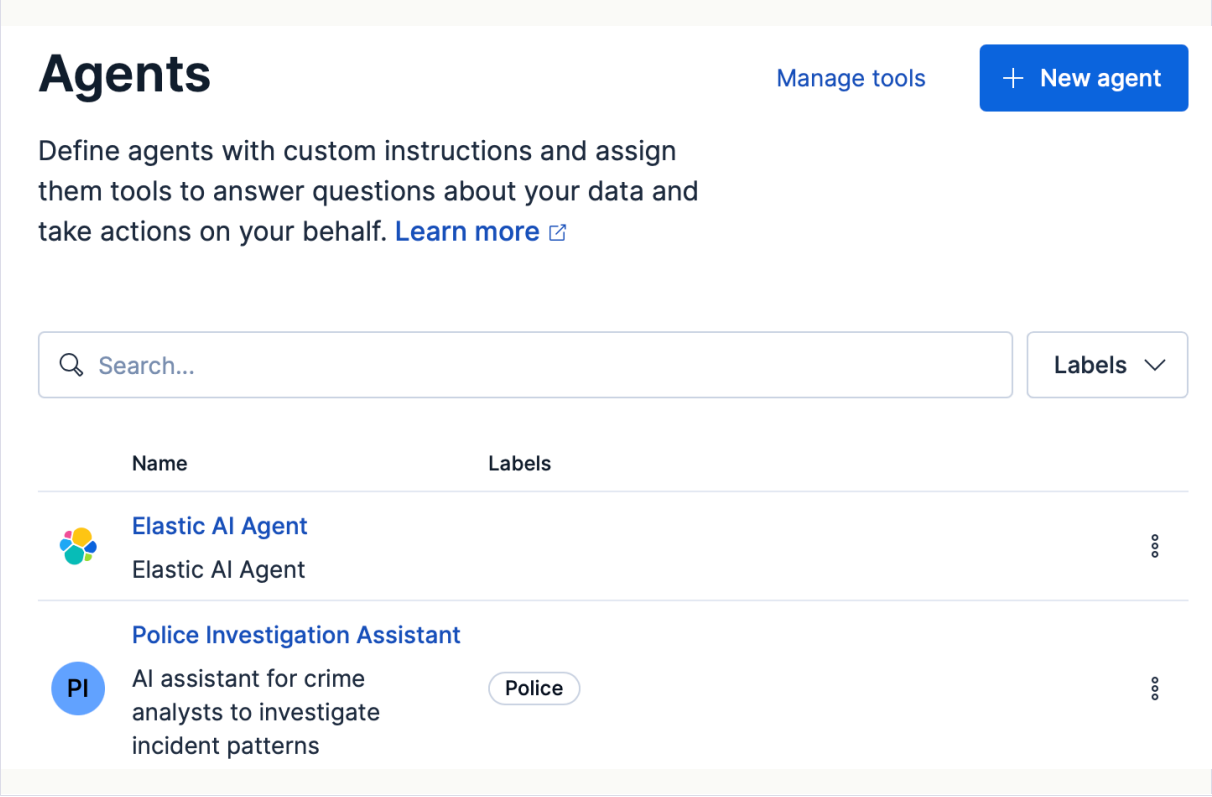


Fig. 9 — Kibana Agents: Police Investigation Assistant deployed alongside Elastic AI Agent

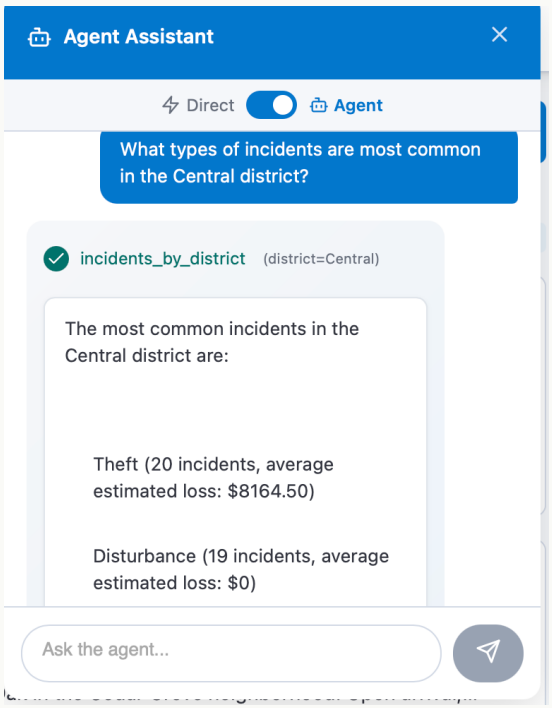


Fig. 10 — Agent calling incidents_by_district tool (district=Central) and returning real structured data

Key Decisions & What I Learned

Why combine search types?

Keyword search is great at finding exact matches but falls apart with synonyms or paraphrasing. Meaning-based search understands intent but can miss precise terms. Combining them gives you the strengths of both — and a document that scores well in both lists naturally rises to the top.

Why generate AI embeddings inside Elasticsearch?

Using an external service to generate AI embeddings costs money per request and can hit rate limits when loading large datasets. By keeping it inside Elasticsearch, embeddings were created automatically for free as each record was saved — with no risk of inconsistency.

Why does data structure matter so much?

Every feature we built — filtering, AI search, summaries, and the agent's tools — was only possible because the right fields were set up from the beginning. Getting the structure right upfront meant nothing had to be rebuilt later to add AI capabilities.

When to use an agent vs. a fixed pipeline?

A fixed RAG pipeline works well for predictable tasks like "summarize this incident." An agent is better when users ask varied questions that require different types of lookups — it can decide on the fly which tool to use and chain multiple steps together.

Key Takeaways

Your AI is only as good as your search

Every answer the language model produced — every summary, every agent response — was based on what Elasticsearch retrieved first. If the search found the wrong documents, the AI confidently gave the wrong answer. Getting search right isn't a nice-to-have; it's the foundation.

Switching AI models should be a setting, not a project

Swapping from one language model to another (Llama-4 to Claude) required changing a single configuration in Kibana — no code changes at all. Claude turned out to follow instructions more reliably for tool-based tasks. Building AI systems this way means you can upgrade as better models become available without reworking your app.

Generating AI embeddings in-house saves real money and headaches

Using a third-party API to generate AI embeddings means paying per record, hitting rate limits during large uploads, and risking inconsistent results if the service changes. Keeping it inside Elasticsearch eliminated all three problems from the start.

Two types of search, two types of questions

Structured queries (counts, totals, breakdowns by district) need precise database lookups. Open-ended questions (find incidents involving a specific scenario) need meaning-based search. The agent used both and picked the right one automatically — that combination is what made it genuinely useful.